

Trilha de Node.js

Módulo 03: Integração com MySQL

Instruções para a melhor prática de Estudo

1. **Leia atentamente todo o conteúdo:** Antes de iniciar qualquer atividade, faça uma leitura detalhada do material fornecido na trilha, compreendendo os conceitos e os exemplos apresentados.
2. **Não se limite ao material da trilha:** Utilize o material da trilha como base, mas busque outros materiais de apoio, como livros, artigos acadêmicos, vídeos, e blogs especializados. Isso enriquecerá o entendimento sobre o tema.
3. **Explore a literatura:** Consulte livros e publicações reconhecidas na área, buscando expandir seu conhecimento além do que foi apresentado. A literatura acadêmica oferece uma base sólida para a compreensão de temas complexos.
4. **Realize todas as atividades propostas:** Conclua cada uma das atividades práticas e teóricas, garantindo que você esteja aplicando o conhecimento adquirido de maneira ativa.
5. **Evite o uso de Inteligência Artificial para resolução de atividades:** Utilize suas próprias habilidades e conhecimentos para resolver os exercícios. O aprendizado vem do esforço e da prática.
6. **Participe de debates:** Discuta os conteúdos estudados com professores, colegas e profissionais da área. O debate enriquece o entendimento e permite a troca de diferentes pontos de vista.
7. **Pratique regularmente:** Não deixe as atividades para a última hora. Pratique diariamente e revise o conteúdo com frequência para consolidar o aprendizado.
8. **Peça feedback:** Solicite o retorno dos professores sobre suas atividades e participe de discussões sobre os erros e acertos, utilizando o feedback para aprimorar suas habilidades.

Essas instruções são fundamentais para garantir um aprendizado profundo e eficaz ao longo das trilhas.

Módulo 3: Integração com MySQL em Node.js

Objetivo do Módulo

Ensinar como integrar aplicações Node.js com o banco de dados MySQL, realizando operações **CRUD (Create, Read, Update, Delete)** de forma **assíncrona, segura e organizada**, seguindo boas práticas adotadas no mercado.

Ao final deste módulo, o aluno será capaz de:

- Configurar o ambiente Node.js + MySQL
- Conectar aplicações Node.js ao MySQL
- Executar consultas SQL de forma assíncrona
- Implementar operações CRUD
- Proteger a aplicação contra SQL Injection
- Utilizar pool de conexões
- Organizar o código em módulos reutilizáveis

Aula 3.1: Configuração do ambiente para MySQL e Node.js

Requisitos para Integração

- Node.js (versão LTS)
- MySQL Server configurado e em execução
- Editor de código: **Visual Studio Code**
- MySQL Workbench (opcional, recomendado)

Instalação e Configuração do MySQL

1. Baixe o **MySQL Community Server**
2. Configure:
 - Usuário (ex: **root** ou outro usuário dedicado)
 - Senha
 - Porta padrão: **3306**
3. Crie um schema para o projeto:

```
CREATE DATABASE projeto_node;
```

Inicialização do Projeto Node.js

```
mkdir projeto-mysql  
cd projeto-mysql  
npm init -y
```

```
npm install mysql2 dotenv
```

🔗 Por que mysql2?

- Suporte nativo a Promises
- Melhor performance
- Compatível com async/await

Aula 3.2: Conectando o Node.js ao MySQL

Configurações do `.env`:

```
DB_HOST=localhost  
DB_USER=root  
DB_PASS=sua_senha  
DB_NAME=projeto_node  
DB_PORT=3306
```

Criação de Conexão

- Exemplo de conexão básica usando as configurações do `.env`:

```
require('dotenv').config();  
const mysql = require('mysql2');  
  
const connection = mysql.createConnection({  
  host: process.env.DB_HOST,  
  user: process.env.DB_USER,  
  password: process.env.DB_PASS,  
  database: process.env.DB_NAME,  
  port: process.env.DB_PORT  
});  
  
connection.connect((err) => {  
  if(err) {  
    console.error('❌ Erro ao conectar ao MySQL:', err.message);  
    return;  
  }  
  
  console.log('✅ Conectado ao MySQL com sucesso!');  
});  
  
module.exports = connection;
```

Agora basta rodar o comando para testes: **node conexao.js**

Aula 3.3: Executando Consultas no MySQL via Node.js

Estrutura Básica de Consultas

1. Exemplo de SELECT:

```
connection.query('SELECT * FROM usuarios', (err, results) => {  
  if(err) {  
    console.error('Erro ao executar consulta:', err);  
    return;  
  }  
  
  console.log('Resultados:', results);  
});
```

Consultas Assíncronas

1. Use `mysql2/promise`:

```
async function main() {  
  const connection = await mysql.createConnection({  
    host: process.env.DB_HOST,  
    user: process.env.DB_USER,  
    password: process.env.DB_PASS,  
    database: process.env.DB_NAME,  
    port: process.env.DB_PORT  
  });  
  
  console.log('✅ Conectado ao MySQL com sucesso!');  
  
  const [rows] = await connection.execute('SELECT * FROM usuarios');  
  console.log(rows);  
  
  await connection.end();  
}  
  
main().catch((err) => {  
  console.error('❌ Erro:', err.message);  
});
```

Manipulação de Erros

- Adicione tratamento de erros para conexões e consultas.

Aula 3.4: Implementando Operações CRUD no Node.js com MySQL

Inserindo Dados (Create)

```
const usuario = { nome: 'João', email: 'joao@email.com' };

connection.query(
  'INSERT INTO usuarios (nome, email) VALUES (?, ?)', [usuario.nome, usuario.email],
  (err) => {
    if(err) {
      console.error('Erro ao inserir dados:', err);
      return;
    }

    console.log('Usuário inserido com sucesso!');
  }
);
```

Consultando Dados (Read)

- Consulta simples:

```
//Read
connection.query('SELECT * FROM usuarios', (err, results) => {
  if(err) throw err;
  console.log(results);
});
```

Atualizando Dados (Update)

```
// Update
connection.query(
  'UPDATE usuarios SET nome = ? WHERE id = ?', ['Carlos', 1],
  (err) => {
    if(err) throw err;
    console.log('Usuário atualizado com sucesso!');
  }
);
```

Excluindo Dados (Delete)

```
// Delete
connection.query(
  'DELETE FROM usuarios WHERE id = ?', [1],
  (err) => {
    if(err) throw err;
    console.log('Usuário excluído com sucesso!');
  }
);
```

Resultado das execuções

```
✓ Conectado ao MySQL com sucesso!
Usuário inserido com sucesso!
Usuário atualizado com sucesso!
[ { id: 2, nome: 'João', email: 'joao@email.com' } ]
Usuário excluído com sucesso!
```

Uso de Prepared Statements

- Proteção contra SQL Injection:

```
const [rows] = await connection.query(  
  'SELECT * FROM usuarios WHERE email = ?', ['joao@email.com']  
);
```

Aula 3.5: Conexão com Pool e Boas Práticas

Conexão com Pool

```
import 'dotenv/config';  
import mysql from 'mysql2/promise';  
  
async function main() {  
  const pool = mysql.createPool({  
    host: process.env.DB_HOST,  
    user: process.env.DB_USER,  
    password: process.env.DB_PASS,  
    database: process.env.DB_NAME,  
    port: process.env.DB_PORT,  
    waitForConnections: true,  
    connectionLimit: 10  
  });  
  
  const [rows] = await pool.query('SELECT * FROM usuarios');  
  console.log(rows);  
}  
  
main().catch(err => {  
  console.error('Erro:', err.message);  
});
```

Estruturação do Código em Módulos

- Organize conexões e consultas em arquivos separados:

```
src/  
├─ db/  
│   └─ db.js  
├─ repositories/  
│   └─ usuarios.js  
├─ index.js  
└─ .env
```

db.js:

```
import 'dotenv/config';
import mysql from 'mysql2/promise';

export const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  database: process.env.DB_NAME,
  port: process.env.DB_PORT,
  waitForConnections: true,
  connectionLimit: 10
});
```

usuarios.js:

```
import { pool } from '../db/db.js';

export async function listarUsuarios() {
  const [rows] = await pool.query('SELECT * FROM usuarios');

  console.log(rows);

  return rows;
}

listarUsuarios();
```

Logs e Monitoramento

- Use pacotes como **winston** ou **pino** para registrar logs.
- Configure alertas para erros críticos.

Lista de Exercícios

Questões Teóricas

1. Explique a diferença entre **mysql2** e **mysql2/promise**.
2. Quais as vantagens de usar variáveis de ambiente na conexão com o banco de dados?
3. O que é um prepared statement e como ele ajuda na segurança da aplicação?
4. Por que é recomendável usar conexões em pool em aplicações Node.js?
5. Descreva o papel do módulo **dotenv** em projetos Node.js.
6. Qual é a estrutura básica para realizar uma consulta **SELECT** com o pacote **mysql2**?
7. Liste três boas práticas ao conectar o Node.js com o MySQL.

8. Explique o conceito de transações no MySQL e como ele pode ser implementado em Node.js.
9. Quais erros podem ocorrer ao conectar ao MySQL, e como tratá-los no Node.js?
10. Por que é importante organizar o código em módulos ao trabalhar com bancos de dados?